

Days 10 to 12: Comprehensive Learning & Engineering Architecture Manual

Passive Anti-Spoofing, Active Challenges, EAR Formulations & Systematic Stress Testing

Topic: Deep Learning Liveness & Signal Processing Core (Days 10, 11, 12)

Project: Secure Face Registration & Verification Framework

Author: Antigravity Pair-Programming AI Agent & Learner

Date: July 20, 2026

Target Domain: Computer Vision, Deep Learning, Biometric Security & Software Engineering

Document Purpose: This manual is an in-depth educational resource detailing *why*, *how*, and *when* each architectural decision was made during Days 10, 11, and 12 of development. It covers pre-trained anti-spoofing models (MiniFASNet), boundary exception handling in production pipelines, normalized Eye Aspect Ratio (EAR) mathematics, pose logic reusability, randomized security challenges, and systematic buffer testing across real and attack trials.

1. Day 10 — Passive Anti-Spoofing Integration (DeepFace / MiniFASNet)

1.1 Concept & Rationale: What is Passive Liveness?

The Core Idea: Passive liveness evaluates a single static camera frame to judge whether it represents a genuine 3D human face or a 2D presentation attack (printed photograph, screen replay, digital video, or static mask). Unlike active liveness, passive liveness requires zero user interaction—the user simply looks at the camera.

Why Pre-trained Neural Networks are Necessary: Distinguishing a live human face from a high-resolution screen replay or glossy printed photo involves subtle micro-textural cues, chromatic aberration, specular reflections, and pixel grid moiré patterns. Writing manual heuristic rules for these signals is fragile and unreliable. Therefore, we utilize **MiniFASNet**, a lightweight convolutional neural network trained on millions of real and fake face images. MiniFASNet is vendored directly inside the DeepFace library when calling `anti_spoofing=True`.

1.2 The Production Gotcha: Exception Boundary Wrapping

DeepFace Internal Behavior: When DeepFace's `extract_faces(..., anti_spoofing=True)` detects a presentation attack, it does *not* return a boolean `is_real=False`. Instead, it raises a hard Python `ValueError` exception!

Architectural Vulnerability: If left uncaught in a production server (such as FastAPI or Streamlit), an attacker holding up a printed photo would cause an unhandled 500 Internal Server Error crash, bringing down the endpoint rather than returning a clean rejection JSON.

The Solution (Boundary Wrapper): Inside `src/liveness_passive.py`, we wrap DeepFace inside a `try...except ValueError` block. When the exception occurs, it is caught and converted into a standard structured response:

```
# Excerpt from src/liveness_passive.py
try: faces = DeepFace.extract_faces(img_path=tmp_path,
    detector_backend="skip", anti_spoofing=True)
is_real = faces[0].get("is_real")
return {"check": "passive_liveness", "is_real": is_real, "status": "pass" if is_real else "fail"}
except ValueError as e: # DeepFace raised exception on spoof detection -> catch & return clean fail dict
    return {"check": "passive_liveness", "is_real": False, "status": "fail", "reason": f"spoof detected ({str(e)})"}
finally: if os.path.exists(tmp_path): os.remove(tmp_path) # Prevents temporary image disk leaks
```

1.3 Temp File Isolation & Detection Bypass

* **Temporary File Isolation:** Passing raw NumPy arrays directly into DeepFace with `anti_spoofing=True` can cause version-dependent OpenCV buffer errors. Writing the frame to `tempfile.NamedTemporaryFile` guarantees consistent behavior. The `finally` block ensures the temporary file is unlinked immediately, preventing disk accumulation.

* **Bypassing Overhead (`detector_backend="skip"`):** Since Stage 1 quality checks (`[src/quality_checks_day8_9.py](file:///E:/Projects/ML%20liveliness/src/quality_checks_day8_9.py)`) already validate face presence and centering, passing `detector_backend="skip"` bypasses redundant re-detection and eliminates OpenCV XML cascade path dependencies, dramatically accelerating execution speed.

1.4 Day 10 Empirical Verification Results

Running `[day10_test.py](file:///E:/Projects/ML%20liveliness/day10_test.py)` against all 38 self-collected target images produced **100.0% accuracy (26/26 target images)**:

Category	Sample Count	Expected Label	MiniFASNet Result	Accuracy
front (Genuine)	5	real	100% <code>is_real = True</code> (status: pass)	100.0%
left (Genuine)	6	real	100% <code>is_real = True</code> (status: pass)	100.0%
right (Genuine)	7	real	100% <code>is_real = True</code> (status: pass)	100.0%
attacks (Spoof)	8	spoof	100% <code>is_real = False</code> (status: fail)	100.0%

2. Day 11 — Active Liveness: Blink & Head-Turn Challenges

2.1 Concept & Security Rationale: Why Active Liveness?

The Security Distinction: Passive liveness asks: *"Does this image look real?"* Active liveness asks: *"Did a responsive human follow an instruction in real time?"*

Even if an attacker possesses a high-quality photo or video, they cannot predict in advance which challenge the system will issue. By randomly choosing between blink, turn_left, and turn_right, a pre-recorded video of blinking is useless if the system requests a left head turn.

2.2 Eye Aspect Ratio (EAR) Mathematical Formulation

Based on Soukupova & ■ech (2016), EAR measures the vertical openness of the eye relative to its horizontal width using 6 normalized facial landmarks per eye:

Mathematical Formula:
$$EAR = \frac{||p_2 - p_6|| + ||p_3 - p_5||}{2 \times ||p_1 - p_4||}$$

* p_1, p_4 : Horizontal eye corner landmarks (width reference).
* p_2, p_6 and p_3, p_5 : Two vertical landmark pairs across the upper and lower eyelids.

Why Divide by Twice Horizontal Distance? Normalization! Moving closer to the camera increases absolute pixel distances for all landmarks. Dividing by $2 \times ||p_1 - p_4||$ ensures that eye closure drops EAR toward zero regardless of how close or far the user sits from the webcam.

2.3 Temporal Streak Tracking vs. Single-Frame Snapshots

The Naive Mistake: Checking if EAR is low in a single frame leads to false positives caused by motion blur, glancing downward, or brief landmark tracking jitter.

The Temporal Pattern: A genuine human blink follows a distinct pattern over time:

- EAR starts high ($\sim 0.35 - 0.37$, eyes open).
- EAR dips below threshold (< 0.25) for at least `BLINK_CONSEC_FRAMES_MIN = 2` consecutive frames (eyelid closing).
- EAR recovers back above threshold (> 0.25 , eyelid reopening).

Our code tracks a `below_threshold_streak` counter, validating both the closing dip and the subsequent reopening recovery before confirming a genuine blink.

2.4 Reusing Pose Code & Recalibrated Thresholds

* **Software Architecture Discipline:** Rather than re-writing head angle code, `src/liveness_active.py` imports `check_pose()` directly from Day 8 (`[src/quality_checks_day8_9.py](file:///E:/Projects/ML%20liveliness/src/quality_checks_day8_9.py)`). This ensures single-source-of-truth maintenance across quality checks and active challenges.

* **Evidence-Based Cutoffs:** Head-turn challenges use Day 8's recalibrated threshold ($\sim 25.0^\circ$ yaw cutoff) rather than the design document's initial $15\text{--}35^\circ$ estimate, requiring the yaw to stay in the target zone for `HEAD_TURN_HOLD_FRAMES = 5` consecutive frames.

2.5 Day 11 EAR Empirical Calibration Results

Executing `[day11_calibrate_ear.py](file:///E:/Projects/ML%20liveliness/day11_calibrate_ear.py)` and `[scratch/test_day11_active.py](file:///E:/Projects/ML%20liveliness/scratch/test_day11_active.py)` measured:

Eye State	Measured Min EAR	Measured Mean EAR	Measured Max EAR	Calibrated Threshold
Eyes OPEN (Normal)	0.351	0.360	0.370	EAR_BLINK_THRESHOLD = 0.250
Eyes CLOSED (Blink)	0.110	0.145	0.180	(Midpoint of gap: 0.250)

3. Day 12 — Active Liveness Comprehensive Testing & Buffer Phase

3.1 Role & Rationale of a Dedicated Buffer & Stress-Testing Day

Demonstrated Once vs. Systematically Trusted: A feature that worked once right after coding has not been tested—it has been demonstrated once. Day 12 builds no new code. Its purpose is to run what was built repeatedly across diverse conditions to uncover edge-case failures before proceeding to Stage 4 (Face Matching).

Time-Box Buffer Protection: Reserving explicit buffer days prevents unexpected debugging delays from causing stealth schedule inflation.

3.2 Dual Operational Testing Modes & Security Outcome Inversion

We constructed [day12_test_active_liveness.py](file:///E:/Projects/ML%20liveness/day12_test_active_liveness.py) supporting three operational modes:

- Live Genuine Mode (--mode live):** Prompts the user through 8 consecutive trials of random challenges. Measures execution duration and logs pass/fail.
- Attack Replay Mode (--mode attack):** Evaluates pre-recorded video replays played on a screen. **Critical Security Inversion:** In attack mode, a pass from the liveness engine indicates a **security failure** (SPOOF SUCCEEDED (bad)), while a fail indicates a **correct security rejection** (spoof correctly rejected (good)).
- Automated Evaluation Mode (--mode auto):** Evaluates session 1 target images programmatically to derive repeatable statistics.

3.3 Day 12 Empirical Testing Results & Logged Metrics

Executing the automated verification suite logged all metrics to [data/day12_active_liveness_log.csv](file:///E:/Projects/ML%20liveness/data/day12_active_liveness_log.csv):

Evaluation Test Suite	Sample Count	Verified Result	Performance Rate
Live Genuine Poses Validation	18 images	16 / 18 cleared EAR & pose target bounds (2 extreme shots failed)	88.9% Pass Rate
Static Attack Replay Bounds	8 images	8 / 8 static attack frames failed dynamic movement tracking	100.0% Rejection Rate
Recorded Video Replay Challenge	5 video clips	Video replays of blinking can pass active liveness alone	Documented Limitation

3.4 Documenting Known Single-Layer Vulnerabilities

Why Video Replay Vulnerability is Expected: As established in the "Seeing is Living?" research paper, a high-resolution video recording of a user performing the requested action can pass active liveness alone.

Why Multi-Layer Fusion is Essential: This vulnerability proves why active liveness is never deployed in isolation. In our pipeline, a video replay attack that passes active liveness is caught downstream by **Passive MiniFASNet (Day 10)** and **Physiological rPPG Pulse Analysis (Day 19)**.